

XIAMEN UNIVERSITY MALAYSIA
厦門大學馬來西亞分校

枚举算法

Enumeration: Brute Force, Bitmask, and Permutations

作者：Wu Youfan
School of Computing and Data Science, Xiamen University Malaysia

2026 年 10 月 18 日

目录

1. 枚举概述
2. 基础枚举模式
3. 多重循环枚举
4. 子集枚举
5. 排列枚举
6. 总结

目录

1 枚举概述

2 基础枚举模式

3 多重循环枚举

4 子集枚举

5 排列枚举

6 总结

什么是枚举

枚举：在有限的候选集合中逐一检查，找到满足条件的解。

枚举三要素

- 枚举范围：候选集合是什么，如何不重不漏地遍历
- 判断条件：如何验证一个候选是否满足要求
- 答案收集：计数、记录下标、提前退出

常见枚举形态

- 单层循环：计数、极值
- 多重循环：二维、三维约束问题
- 子集枚举：位掩码
- 排列枚举：`next_permutation`

枚举 = 一个范围 + 一个判断 + 一次记录

目录

1 枚举概述

2 基础枚举模式

3 多重循环枚举

4 子集枚举

5 排列枚举

6 总结

素数判定

问题：判断整数 x 是否为素数。

枚举思路

- 素数定义：大于 1 且只能被 1 和自身整除
- 枚举因子：从 2 枚举到 $\sqrt{x}$
- 判断条件：一旦存在因子，立即返回 false

```
1 bool isPrime(int x)
2 {
3     for (int i = 2; i * i <= x; i++)
4     {
5         if (x % i == 0)
6             return false;
7     }
8     return true;
9 }
```

为什么枚举到 $\sqrt{x}$ 就够：若 $x = a \times b$ 且 $a \leq b$，则必然有 $a \leq \sqrt{x}$。

极值查找与 INF 技巧

问题：在数组 `a[1..n]` 中找到最大值与最小值。

INF 常量

```
1 #define INF   0x3f3f3f3f              // int
2 #define LLINF 0x3f3f3f3f3f3f3f3fLL    // long long
```

初始化策略

- 求最大值：`mx = -INF`，保证第一个元素一定能更新
- 求最小值：`mn = INF`，保证第一个元素一定能更新
- 不能初始化为 0：数组全为负数时 0 会错当最大值

最大值与最小值

极值查找与 INF 技巧


```
int mx = -INF, id = -1;
for (int i = 1; i <= n; i++)
    if (mx < a[i]) { mx = a[i]; id = i; }

int mn = INF;
for (int i = 1; i <= n; i++)
    mn = min(mn, a[i]);
```

选 INF 的理由：`0x3f3f3f3f` 约等于 10^9，既够大又不会让加法溢出。

严格比较与相等比较

问题：找出最大值下标，但数组中有多处最大值时如何处理？

取第一个最大值

```
1 if (mx < a[i]) { mx = a[i]; id = i; }
```

- 严格小于：只有严格更大的值才更新
- 第一次遇到最大值时记录
- 之后遇到相同的值，不满足小于条件，不再更新

取最后一个最大值

```
1 if (mx <= a[i]) { mx = a[i]; id = i; }
```

- 小于等于：等于时也更新
- 最终留下的是最靠后的那个下标

差一个等号，差一个下标

标志变量与跳出多层循环

问题：多重循环中找到答案后如何提前退出？

C++ 的 `break` 只跳出最内层，无法一次性跳出多层。

通用模式：标志变量

```
bool flag = false;
for (int i = 1; i <= n; i++)
{
    for (int j = 1; j <= m; j++)
    {
        for (int k = 1; k <= p; k++)
        {
            if (isWanted(i, j, k))
            {
                flag = true;
                break;                  // 跳出 k 循环
            }
        }
```

标志变量与跳出多层循环

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

```
14            if (flag) break;              // 跳出 j 循环
15        }
16        if (flag) break;                  // 跳出 i 循环
17    }
```

要点

- 每层循环结束都要检查 `flag`
- 层数越多，代码越冗长
- 可用函数封装，用 `return` 代替 `break`

目录

1 枚举概述

2 基础枚举模式

3 多重循环枚举

4 子集枚举

5 排列枚举

6 总结

例题：鸡兔同笼

题目：笼中有鸡和兔若干只，共有 head 个头、foot 条腿。问鸡和兔各几只？

示例 1

- **输入**：head = 10，foot = 32
- **输出**：4 6

示例 2

- **输入**：head = 20，foot = 56
- **输出**：12 8

示例 3

- **输入**：head = 5，foot = 20
- **输出**：No solution.

鸡兔同笼：解法

枚举思路

- 枚举鸡的数量 i，范围 1 到 100
- 枚举兔的数量 j，范围 1 到 100
- 判断条件：$i + j = head$ 且 $2i + 4j = foot$

```
bool flag = false;
int ansi, ansj;
for (int i = 1; i <= 100; i++)
{
    for (int j = 1; j <= 100; j++)
    {
        if (i + j == head && i * 2 + j * 4 == foot)
        {
            ansi = i;
            ansj = j;
            flag = true;
            break;
```

鸡兔同笼：解法


```
13          }
14      }
15      if (flag) break;
16  }
17
18  if (flag) cout << ansi << " " << ansj << endl;
19  else      cout << "No solution." << endl;
```

优化方向：j 可由 $head - i$ 直接算出，$O(n^2)$ 可以降到 $O(n)$。

目录

1 枚举概述

2 基础枚举模式

3 多重循环枚举

4 子集枚举

5 排列枚举

6 总结



子集枚举：位掩码思想

问题：枚举 n 个元素的所有子集（共 2^n 个）。

核心思想：**用二进制位表示选 / 不选**

- 第 i 位为 1 表示选第 i 个元素
- 第 i 位为 0 表示不选
- 所有子集 $=$ 所有 n 位二进制数

以 $n = 3$ 为例

- `000`：空集
- `001`：只选第 0 个元素
- `011`：选第 0、1 个元素
- `111`：全选

子集枚举 $=$ 从 0 枚举到 $2^n - 1$

子集枚举：代码模板


```cpp
for (int mask = 0; mask < (1 << n); mask++)
{
    for (int i = 0; i < n; i++)
    {
        if ((mask >> i) & 1)   // 第 i 位是否为 1
        {
            // 选中第 i 个元素
        }
        else
        {
            // 不选第 i 个元素
        }
    }
}
```

关键位运算

- `1 << n`：2^n

子集枚举：代码模板

- `(mask >> i) & 1`：取出 mask 的第 i 位
- 循环总次数 2^n，每个 mask 内部再扫一遍 n 位

复杂度：$O(n \cdot 2^n)$。

适用条件：n 一般不超过 20 到 25，否则候选数过多。

目录

1 枚举概述

2 基础枚举模式

3 多重循环枚举

4 子集枚举

5 排列枚举

6 总结

排列枚举：next permutation

工具：STL 的 `next_permutation`，可以直接按字典序生成下一个排列。

```
vector<int> v;
for (int i = 1; i <= n; i++)
{
    int x;
    cin >> x;
    v.push_back(x);
}
do
{
    for (int i = 0; i < n; i++)
        cout << v[i] << " ";
    cout << endl;
} while (next_permutation(v.begin(), v.end()));
```

关键要点

- 初始序列必须已经升序，否则会漏掉排列

排列枚举：next permutation

- 返回 true 表示还有下一个排列，返回 false 表示已到最后一个
- 序列含重复元素时会产生重复排列，可用集合去重

复杂度：$O(n!)$ 次全排列，每次 $O(n)$。



目录

1 枚举概述

2 基础枚举模式

3 多重循环枚举

4 子集枚举

5 排列枚举

6 总结

总结：枚举的五个动作

1. **定范围**：候选集合是什么，如何不重不漏
2. **定判断**：一个候选满足什么条件才算解
3. **定记录**：计数、存下标、存答案
4. **定退出**：多重循环用 `flag` + `break`
5. **定优化**：能用一个量推出另一个量时不要双重循环

三种常见枚举

- 极值查找：`mx = -INF`，严格比较取首次
- 子集枚举：mask 从 0 到 $2^n - 1$，位运算判断
- 排列枚举：`next_permutation`，注意初始升序

枚举的本质：不重不漏地检查每一个候选

附录：常见陷阱速查

错误	修复
最大值初始化为 0	改为 `mx = -INF`
最小值初始化为 0	改为 `mn = INF`
取首次最大值写成 $\leq$	改用严格的 $<$
取末次最大值写成 $<$	改用 $\leq$
多重循环只能跳出一层	引入 `flag` 每层检查
子集枚举漏掉空集	mask 从 0 开始
子集枚举越界	mask 上界严格小于 2^n
排列枚举漏掉排列	初始序列必须升序

范围不重不漏，判断严格正确，记录及时更新